

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

Dr. M. Anoop

QUESTIONS: 10

Total Marks:50

Annexure A

Scenario based

Code of Conduct:

I ___Narmatha J (192521341)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Narmatha J

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.

3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2

- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS:

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

Suitable Scheduling Algorithm:

The most suitable scheduling algorithm is Priority Scheduling.

Justification:

- Payment gateway transactions (P1) are very important and should be completed first.
- Seat availability checks (P2) are processed after payment.
- Ticket confirmation and printing (P3) can be executed last.
- This scheduling reduces waiting time for important tasks and improves CPU utilization during heavy traffic.

Sample Execution Order:

Process	Priority	Execution Order
P1 – Payment	High	1st
P2 – Seat Availability	Medium	2nd
P3 – Ticket Confirmation	Low	3rd

Execution Order:

P1 → P2 → P3

Drawback:

A drawback of Priority Scheduling is starvation. Low-priority processes may wait for a long time if high-priority processes keep arriving.

Question 2 – Deadlock in a Banking Transaction Server

Resource Allocation Graph (Text Form):

T1 holds A $\rightarrow$ requests B

T2 holds B $\rightarrow$ requests C

T3 holds C $\rightarrow$ requests A

or

A $\rightarrow$ T1 $\rightarrow$ B

B $\rightarrow$ T2 $\rightarrow$ C

C $\rightarrow$ T3 $\rightarrow$ A

Deadlock Identification:

Yes, a deadlock has occurred.

Reason:

- T1 waits for B.
- T2 waits for C.
- T3 waits for A.
- Each process waits for a resource held by another process, forming a circular wait.

Technique to Fix:

Deadlock Prevention:

- Force all transactions to request resources in the same order.
- This removes the circular wait condition and prevents deadlock.

Locking Order Redesign:

Use the same locking sequence for every transaction.

Example:

First lock A

Then lock B

Finally lock C

This consistent order avoids deadlocks.

Question 3 – Mobile OS App Switching Lag

Why CPU Remains Idle:

- Many applications are waiting for I/O operations like storage and network.
- When processes wait for I/O, the CPU becomes idle.
- Frequent context switching also wastes CPU time.

Time Quantum Analysis:

A 20 ms quantum is too small.

- Too many context switches occur.
- CPU spends more time switching than executing processes.
- This reduces performance.

Recommendation:

Increase the time quantum to 40–50 ms or use Multilevel Feedback Queue (MLFQ) scheduling.

Reason:

- Fewer context switches.
- Better response for interactive applications.
- Improved CPU utilization.

Benefits:

- Faster app switching.
- Better throughput.
- Less CPU overhead.
- Improved user experience.

Question 4 – Deadlock Risk in Operating System Update**Can Deadlock Occur?**

Yes. The given sequence can lead to a deadlock.

Circular Wait Condition:

M1 holds R1 and waits for R2

M2 holds R2 and waits for R3

M3 holds R3 and waits for R1

This creates the cycle:

$M1 \rightarrow R2$

$M2 \rightarrow R3$

$M3 \rightarrow R1$

This satisfies the circular wait condition.

Banker's Algorithm Style Allocation:

Allocate resources only if the system remains in a **safe state**.

Example:

- Grant a resource only when enough resources remain for other modules to complete.
- If not safe, delay the allocation.

This prevents deadlock.

Module Priority:

Suggested priority:

1. M1 – Copy update files
2. M2 – Install drivers
3. M3 – Update registry
4. M4 – Validate system integrity

This order reduces waiting time and improves CPU utilization.

Question 5 – Cloud Server CPU Utilization Analysis

Given:

- I/O wait probability (p) = 0.8
- Number of VMs (n) = 7

Formula:

$$U = 1 - p^n$$

CPU Utilization Calculation:

$$U = 1 - (0.8)^7$$

$$0.8^7 = 0.2097$$

$$U = 1 - 0.2097$$

$$U = 0.7903$$

CPU Utilization

$$U = 79.03\%$$

Answer: CPU Utilization = 79.03%

Interpretation:

- The server is well utilized.
- Approximately 79% CPU utilization indicates that the server resources are being used efficiently without excessive idle time.

Effect of Changing Number of VMs:

- **Increasing VMs**
 - Increases CPU utilization.
 - Improves throughput.
 - Too many VMs may cause overload.
- **Decreasing VMs**
 - Reduces throughput.
 - CPU remains idle for longer periods.

Two OS-Level Strategies:

Strategy 1: Multiprogramming

- Run multiple processes simultaneously.
- CPU works while other processes wait for I/O.

Strategy 2: Efficient CPU Scheduling

- Use Priority Scheduling or Multilevel Feedback Queue.
- Reduces waiting time.
- Improves CPU utilization and overall performance.